

Operating Systems

COMP6697001

Final Project: CPU Scheduling Simulator using Python

May, 2026

Lecturer

Dr. Satrio Pradono Suryodiningrat

Project Overview

In modern operating systems, CPU scheduling plays an important role in managing processes and allocating computing resources efficiently. Different scheduling algorithms provide different trade-offs between waiting time, turnaround time, responsiveness, and fairness.

In this project, you will design and implement a Python-based CPU Scheduling Simulator that demonstrates how operating systems schedule processes using different scheduling techniques.

This project combines:

1. operating system concepts,
2. programming implementation,
3. algorithm analysis,
4. and responsible use of generative AI tools.

You are **encouraged** to use generative AI tools such as ChatGPT, Claude, GitHub Copilot, Gemini, or similar tools to assist your development process. However, you must fully understand and explain all submitted work.

Project Objectives

The objective of this project is to help students:

- understand the behavior of CPU scheduling algorithms,
- compare the efficiency of scheduling techniques,
- analyze process management and resource allocation,
- apply operating system concepts in a practical implementation,
- and critically evaluate the use of AI-assisted programming tools.

Upon completing this project, students are expected to:

- classify scheduling techniques according to their principles,
- compare scheduling algorithm performance,
- analyze process execution behavior,

- and evaluate the strengths and weaknesses of different scheduling methods.

Programming Requirements

You must use:

- Python 3.x

You may use:

- Tkinter
- Matplotlib
- Pandas
- Standard Python libraries

You may NOT use:

- prebuilt CPU scheduling simulators,
- copied projects from GitHub,
- fully AI-generated submissions without modification and understanding.

Due Date

Sunday 21 June 2026 @ 11.59 PM

Submission Requirement

You must submit:

1. Source Code

Submit all Python source files.

2. Project Report

The report should include:

- Introduction
- Scheduling algorithm explanation
- Screenshots

- Performance comparison
- Discussion and analysis
- Conclusion

3. Reflection and AI Usage Report

Maximum 2 pages.

4. Demo Video

Maximum 10 minutes.

The demo should include:

- Program execution
- Feature explanation
- Algorithm comparison
- Brief explanation of AI usage

Important Notes

- This is a group project consisting of 2 students per group.
- All submissions must be original.
- You are responsible for understanding every line of submitted code.
- Similar or duplicated submissions between groups may receive zero marks.
- AI-generated code must be clearly disclosed in the reflection report.

Each group must clearly explain:

1. which parts used AI assistance,
 2. which parts were written manually,
 3. how AI suggestions were modified or verified.
- Failure to disclose AI usage honestly may be treated as academic misconduct.
 - Late submissions may not be accepted unless approved officially.

Part 1 - Core CPU Scheduling Simulator (60 Marks)

For this part of the assessment, you will implement a CPU Scheduling Simulator supporting the following algorithms:

Required Algorithms

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Round Robin (RR)
- Priority Scheduling

Your program must allow users to input:

- Process ID
- Arrival Time
- Burst Time
- Priority Value

Your simulator must generate:

- Execution Order
- Waiting Time
- Turnaround Time
- Average Waiting Time
- Average Turnaround Time

Your simulator must also display:

- A simple Gantt Chart visualization

Part 2 - Advanced Features and Analysis (20 Marks)

For this part of the assessment, you will extend your simulator with additional features.

You may choose one or more of the following:

- Add a graphical user interface (GUI)
- Add real-time process simulation
- Add process generation using random workloads
- Compare algorithm performance automatically
- Add multilevel queue scheduling
- Add shortest remaining time first (SRTF)
- Add performance statistics visualization
- Add export functionality for reports/results

You may also propose your own additional feature(s).

Part 3 - Reflection and AI Usage Report (20 Marks)

You will submit a short reflection document together with your source code.

This document must clearly explain:

AI Usage Disclosure

- What AI tool or tools you used
- What prompts you provided
- Which parts of the code were AI-assisted
- Which parts were fully written by yourself
- How you modified or improved AI-generated code
- How you verified the correctness of the generated code

Reflection

- Where AI was helpful
- Where AI gave incorrect or poor suggestions
- Challenges faced during implementation
- What you learned from this project
- Which scheduling algorithm you believe performs best and why

Failure to disclose AI usage honestly may be treated as academic misconduct.

Marking Scheme

Component	Marks
Core Scheduling Algorithms	35
Correctness and Functionality	25
Advanced Features	20
Reflection and AI Usage Report	10
Code Quality and Documentation	5
Report and Demo Video	5
Total	100